
EVALUATION OF 3D GAUSSIAN SPLATTING FOR REAL-TIME SCENE RECONSTRUCTION *

Anish Nalla
analla@asu.edu

Neha Kashyap
nkashya6@asu.edu

Mawadda Abuhannoud
mabuhann@asu.edu

Cephus Prabhuchristopher Agassi
cagassi@asu.edu

Satyam Chaurasia
schaura1@asu.edu

ABSTRACT

This project explores 3D Gaussian Splatting (3DGS), a modern rendering technique for real-time 3D scene reconstruction from 2D imagery. Traditional approaches such as Neural Radiance Fields (NeRFs) achieve impressive visual fidelity but are often computationally intensive and slow during rendering. In contrast, 3DGS dramatically improves both training and inference speed. Our goal is to analyze the trade-offs between storage efficiency, geometric regularization, and visual fidelity by modifying the core training parameters of the 3DGS pipeline. Using the Brush interface and the gsplat backend, we conducted an evaluation of three distinct configurations on the standard NeRF synthetic Lego dataset: Baseline (default pre-optimized configuration), Compact, and Regularized. Our results demonstrate that targeted parameter tuning can significantly optimize the pipeline. The "Regularized" configuration achieved superior visual fidelity compared to the baseline while reducing file size by 25.6%, effectively mitigating overfitting and visual artifacts. Conversely, the "Compact" achieved a massive 97.2% reduction in file size with a trade-off in high-frequency detail. These findings provide a blueprint for optimizing Gaussian Splatting for resource-constrained environments versus high-fidelity visualization.

1 Introduction

The field of 3D computer vision has seen steady progress in recent years, specifically the photorealistic capture and synthesis of real-world scenes. Prior methods such as Neural Radiance Fields (NeRFs) successfully leveraged deep learning to represent complex scenes with high fidelity. However, due to the nature of NeRFs, where the scene is encoded within the weights of a multilayer perceptron, the rendering speed is inherently limited. This is because every pixel requires ray marching and network querying.

This project is centered on the more recent method of 3D Gaussian Splatting (3DGS), which provides a novel and highly efficient scene representation that bypasses the limitations of previous implicit methods. By modeling scenes using millions of anisotropic 3D Gaussians, 3DGS enables high-quality rendering using a highly parallelized and differentiable rasterization pipeline [1]. This approach has demonstrated rendering speeds exceeding 100 frames per second for complex scenes, which position it as a potential breakthrough in technology for applications that need real-time interaction.

Our project focuses on evaluating the efficiency and quality trade-offs inherent in the 3DGS architecture. Although our initial scope considered a ground-up implementation, the project evolved to leverage the gsplat [2] library and the Brush [3] interface to perform an analysis of training parameters. Our primary objective is to investigate algorithmic configurations that minimize the limitations of the core 3DGS architecture, specifically targeting the high memory footprint and the prevalence of visual artifacts known as "floaters". We aim to benchmark distinct parameter profiles, ranging from highly compact representations to geometrically regularized models, to determine how specific training hyperparameters affect the balance between storage size, convergence speed, and visual fidelity.

**Citation:* Authors. Title. Pages.... DOI:000000/11111.

2 Problem Statement

The field of neural rendering has achieved great progress in reconstructing and visualizing 3D scenes from 2D image datasets. Techniques such as Neural Radiance Fields (NeRFs) have shown that deep learning can generate highly photorealistic 3D representations by learning continuous volumetric scene functions. However, despite their visual fidelity, NeRFs and similar neural methods remain computationally expensive and inefficient for real-time applications. The need for extensive training, large GPU resources, and slow rendering times limits their use in interactive settings such as virtual reality (VR), augmented reality (AR), and robotics. 3DGS works towards addressing these issues. With 3DGS achieving unprecedented rendering speeds, its explicit nature introduces distinct technical challenges that limit scalability and deployment efficiency. We aim to address the critical issues of high memory footprint and the trade-offs involved in regularization. To demonstrate the potential of 3DGS optimizations, we performed a detailed quantitative comparison of differing training configurations to analyze the specific trade-offs between storage size and reconstruction quality.

2.1 Motivation

The motivation behind this project arises from the growing demand for real-time, high-quality 3D visualization across industries, from entertainment and gaming to autonomous navigation and digital twins. Current neural methods, while powerful, fail to meet the computational efficiency required for these use cases. The emergence of Gaussian Splatting addresses this problem. Rather than relying on implicit neural volumes, Gaussian Splatting represents the scene using explicit 3D Gaussian primitives that encode color, opacity, and spatial position. This approach significantly accelerates rendering while maintaining photorealistic quality. Our goal is to study and analyze this technique to evaluate how specific parameter optimizations can bridge the gap between rendering quality and computational efficiency, specifically for resource-constrained environments.

This method represents a fundamental shift away from implicit representations, making it an important area of study for future real-time graphics and ML integration. Through our experimental analysis, we aim to achieve three objectives. First, identify configuration profiles that effectively reduce the memory burden of 3DGS to accelerate its adoption in low-memory environments. Second, gain practical expertise in the mechanics of the differentiable rasterization pipeline and the impact of spherical harmonic coefficients on file size. Lastly, provide a clear, quantitative benchmark that defines the lower bounds of storage size required to maintain recognizable 3D geometry.

2.2 Technical Background

3D Gaussian Splatting is based on concepts from computer graphics, computer vision, and differentiable rendering. The primary constraint of 3DGS is the significant memory requirement, particularly VRAM consumption and disk storage. A normal scene typically requires the storage of millions of 3D Gaussians, each requiring storage for its center, scaling factor, covariance matrix, opacity, and a large set of spherical harmonic coefficients (SH) [1]. Each of these aspects of the Gaussian contributes to an overall large number of dimensions to handle, and the accumulated memory usage restricts the complexity and physical size of scenes that can be handled effectively on standard GPUs.

The high quality and performance of 3DGS depend on its adaptive density control technique, which is the process of periodically adding Gaussians in areas with high positional gradients and removing Gaussians that minimally contribute to the rendering error. Tuning this system is highly technical and sensitive, specifically when planning how to stop growth, how aggressively to prune, and how to weigh structural similarity. Our experiments focus on manipulating these specific variables to control the output size.

2.3 Related Work

The foundation for modern view synthesis was established by Mildenhall et al. with the introduction of Neural Radiance Fields (NeRF). NeRF represented 3D scenes as a continuous volumetric function parameterized by a multi-layer perceptron (MLP). Given a 3D coordinate and a viewing direction, the network outputs a color and density value. Rendering is achieved through volume rendering integration, which samples hundreds of points per ray through the scene to synthesize a final image [2]. 3D Gaussian Splatting revolutionized the field by introducing explicit Gaussian representations, achieving real-time rendering with comparable or superior visual quality to NeRFs [1].

Our work relates closely to these advancements by utilizing the gsplat library and the Brush software interface to conduct a rigorous analysis of the 3D Gaussian Splatting pipeline. This approach allows us to benchmark reconstruction quality, memory efficiency, and speed across different configuration profiles, validating the effectiveness of regularization and parameter quantization in real-world applications.

3 Methodology

To evaluate the performance trade-offs of 3D Gaussian Splatting, we moved away from a ground-up implementation to leverage an open-source stack. This allowed us to focus our efforts on hyperparameter optimization and comparative analysis rather than low-level kernel debugging. Our methodology is structured around three distinct configuration profiles designed to test the limits of storage efficiency versus visual fidelity.

3.1 Software and Implementation

We utilized Brush, a widely used GUI and management tool for Gaussian Splatting that operates on top of the gsplat library. The gsplat library provides highly optimized CUDA kernels for the differentiable rasterization of 3D Gaussians. Using this backend ensured that our performance metrics were representative of SoTA implementations.

For data analysis and visualization, we developed a custom evaluation pipeline using PyTorch and Matplotlib. This pipeline automated the extraction of rendering metrics at fixed intervals (every 2,000 iterations) during the training process, allowing us to map the convergence curves of different configuration profiles

3.2 Dataset

We conducted our experiments on the NeRF Synthetic dataset, specifically the "Lego" scene. This dataset serves as a standard benchmark in the field of neural rendering, consisting of high-resolution synthetic images with precise camera poses. The Lego scene is particularly valuable for this analysis because it contains complex geometry and occlusions, which rigorously stress-test the adaptive densification and pruning mechanisms of the 3DGS algorithm.

3.3 Experimental Configurations

We designed three distinct parameter profiles to represent different use cases. These profiles vary in their approach to densification (adding points), pruning (removing points), and regularization (penalizing artifacts).

- **Baseline:** This profile mimics the standard configuration typically used in the original 3DGS paper. It prioritizes maximum visual quality, allowing the model to grow to a large number of Gaussians with high-order spherical harmonics (SH Degree 3) to capture view-dependent lighting effects.
- **Compact:** This profile aims to minimize storage footprint. We reduced the Spherical Harmonics degree to 2, significantly cutting the data stored per Gaussian. Furthermore, we implemented an early "Growth Stop" at 7,000 iterations and increased the "Growth Threshold," forcing the model to represent the scene with fewer, more significant primitives.
- **Regularized:** This profile aims to reduce visual artifacts known as "floaters" and prevent overfitting. We introduced a non-zero **Opacity Loss (0.0001)** to force Gaussians toward binary opacity (either fully visible or invisible), clearing up semi-transparent noise. Additionally, we doubled the **SSIM Weight (0.4)** in the loss function to prioritize structural coherence over pixel-perfect noise matching.

The specific hyperparameters for each profile are detailed in Table 1.

Table 1: Hyperparameter Configurations

Parameter	Compact	Regularized	Baseline
SH Degree	2	3	3
Growth Stop (Iter)	7,000	15,000	25,000
Growth Threshold	0.0006	0.0002	0.0002
Refinement Freq	300	100	200
Opacity Loss	0.005	0.0001	0.0 (Default)
SSIM Weight	0.2	0.4	0.2

3.4 Experimental Configurations

To quantitatively assess the reconstruction quality, we employed three standard industry metrics:

1. **PSNR (Peak Signal-to-Noise Ratio):** Measures the pixel-wise error between the rendered image and the ground truth. Higher is better.
2. **SSIM (Structural Similarity Index):** Evaluates the perceived change in structural information (luminance, contrast, and structure). Higher is better.
3. **LPIPS (Learned Perceptual Image Patch Similarity):** Uses a pre-trained VGG network to measure perceptual similarity, which correlates better with human vision than simple pixel difference. Lower is better.

Additionally, we tracked **File Size (MB)** and qualitative visual artifacts to evaluate the efficiency and cleanliness of the resulting models.

4 Results

We evaluated the three configuration profiles on the Lego scene over 30,000 iterations. Our analysis focuses on three key areas: quantitative visual fidelity, convergence behavior, and storage efficiency

4.1 Quantitative Comparison

Table 2 summarizes the final performance metrics for each configuration. Contrary to the initial assumption that higher fidelity settings would yield the best metrics, the Regularized profile achieved the highest scores across all three categories.

Table 2: Final Evaluation Metrics (30k Iterations)

Metric	Compact	Baseline	Regularized	Best Performing
PSNR	22.50 dB	26.42 dB	26.69 dB	Regularized
SSIM	0.807	0.916	0.924	Regularized
LPIPS	0.212	0.100	0.093	Regularized

The Regularized configuration outperformed the Baseline baseline by roughly 0.27 dB in PSNR and 0.008 in SSIM. This improvement indicates that the default Brush settings tend to overfit to high-frequency noise. By increasing the SSIM Weight from 0.2 to 0.4 and introducing an Opacity Loss of 0.0001, the Regularized model was forced to learn structurally coherent geometry rather than relying on semi-transparent artifacts to minimize pixel error.

The Compact model, while significantly behind in visual quality (4 dB in PSNR), still maintained a recognizable structure (SSIM > 0.8), suggesting it successfully captured the general geometry of the scene despite losing high frequency details.

4.2 Training and Convergence

Analyzing the training logs reveals distinct convergence behaviors dictated by the Growth parameters:

- The Compact model’s performance plateaus abruptly after iteration 7,000, meaning the Growth Stop parameter contributed to reduced size. By preventing the addition of new Gaussians early in the training process, the model was forced to optimize only the positions and colors of the existing primitive set. While this capped the visual quality (PSNR $\sim$ 22.5 dB), it ensured the model remained lightweight.
- The Regularized model initially trailed the Baseline model during the first 5,000 iterations. However, it overtook the baseline around iteration 12,000 and continued to improve steadily. This suggests that the Opacity Loss acts as a constraint that initially slows down learning but prevents the model from getting stuck in local minima populated by “floater” artifacts, ultimately leading to a higher quality convergence.

4.3 Efficiency and Storage

The most significant trade-offs were observed in file size and resource usage. Table 3 presents the final storage footprint of the trained models

The Regularized model is approximately 25% smaller than the Baseline model while providing better visual quality. The Opacity Loss successfully pruned unnecessary, semi-transparent Gaussians that usually clutter the scene background, resulting in a cleaner and more efficient representation.

Table 3: Storage Efficiency

Profile	Final File Size	Reduction vs. Baseline
Baseline	11,994 KB (~ 12.0 MB)	-
Regularized	8,922 KB (~ 8.9 MB)	25.6%
Compact	329 KB (~ 0.3 MB)	97.2%

The Compact profile achieved a massive 97% reduction in file size. By lowering the Spherical Harmonics degree to 2 (reducing the number of color coefficients stored per point) and aggressively limiting the Gaussian count via Growth Threshold, we produced a 3D scene representation that fits within 329 KB. While the texture details were blurred, this proves 3DGS can be viable for extremely low-bandwidth applications (e.g., mobile web rendering) where 12 MB per object is unacceptable.

4.4 Qualitative Analysis



(a) Compact



(b) Baseline



(c) Regularized

Visual inspection of the rendered outputs confirms the numerical data. Although the Baseline and Regularized final evaluation images look quite similar, the Baseline’s metrics indicate that it renders some "floaters"—cloudy, semi-transparent artifacts floating near the camera or in empty space, which are characteristic of overfitting in 3DGS, inside the geometry most likely. The Regularized output largely eliminated these artifacts, resulting in solid, well-defined geometry and better metrics. The Compact output appeared softer and lacked specular highlights (view-dependent reflections), a direct consequence of reducing the SH degree from 3 to 2, limiting the model’s ability to represent complex lighting interactions.

5 Conclusion

In this work, we evaluated 3D Gaussian Splatting on the NeRF synthetic "Lego" scene by comparing three training profiles: Baseline, Compact, and Regularized. We utilized a Brush/gsplat pipeline. Our experiments show that small changes in growth, pruning, and regularization can shift the balance between quality and efficiency. In the Regularized profile, we slightly outperformed the default Baseline configuration in PSNR, SSIM, and LPIPS while reducing file size by about 25%, while the Compact profile achieved a roughly 97% reduction in size at the cost of high-frequency detail. Together, these results suggest that 3DGS is not just a fast alternative to NeRF and similar methods, but also an adjustable representation that can either become a cleaner, high-fidelity reconstruction or a highly compressed scene asset, depending on the constraints of the target application.

While this study focused on the internal optimization of the 3D Gaussian Splatting pipeline, future research should expand the comparative analysis to include established implicit representations, particularly Neural Radiance Fields (NeRFs) [4]. A key objective will be to benchmark our "Compact" and "Regularized" profiles against state-of-the-art NeRF implementations. Because NeRFs store scenes in the weights of a neural network rather than as explicit geometric primitives, they often achieve high compression rates naturally. A direct comparison of storage size versus rendering FPS would help determine the precise break-even point where 3DGS becomes more viable than NeRFs for web-based or mobile applications.

References

- [1] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics (TOG)*, 42(4):139–1, 2023.
- [2] Vickie Ye, Ruilong Li, Justin Kerr, Matias Turkulainen, et al. gsplat: An open-source library for gaussian splatting. *Journal of Machine Learning Research*, 2025. GitHub Repository: <https://github.com/nerfstudio-project/gsplat>.
- [3] Arthur Brussee. Brush: Universal gaussian splatting interface. <https://github.com/ArthurBrussee/brush>, 2023. Accessed: 2024.
- [4] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. *Communications of the ACM*, 65(1):99–106, 2021.